



Sky: CCIP DVN Adapter

Security Review

Cantina Managed review by:

Christoph Michel, Lead Security Researcher

M4rio.eth, Lead Security Researcher

June 24, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Low Risk	4
3.1.1	LayerZero confirmation guarantees for CCIP-backed DVNs reduce to Chainlink CCIP finality assumptions	4
3.1.2	Require prefunding the <code>CCIPDVNAdapter</code>	4
3.1.3	Use of deprecated <code>EVMExtraArgsV1</code> for CCIP messages	4
3.1.4	Source-side <code>CCIPDVNAdapter</code> makes CCIP mandatory for every packet emission . . .	5
3.2	Informational	6
3.2.1	DVN adapter vs OApp receive library misconfiguration	6
3.2.2	DVN fees can be discounted below cost breaking bridging longterm	7
3.2.3	Avoid bidirectional setup for one-way Sky Gov bridge	7
3.2.4	Implicit fee quote assumption on message byte contents	7
3.2.5	<code>CCIPDVNAdapterFeeLib</code> ignores configured <code>floorMarginUSD</code>	8
3.2.6	<code>CCIPDVNAdapter</code> normalizes endpoint ids and aliases v1/v2 route configuration . . .	8
3.2.7	CCIP service limits can revert adapter sends	9
3.2.8	<code>CCIPDVNAdapter</code> reverts on any DVN-targeted option	10
3.2.9	CCIP adapter pauser does not pause any CCIP adapter flow	10
3.2.10	Minor issues, Typos & Documentation	10

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Sky Protocol is a decentralised protocol developed around the USDS stablecoin.

From Jun 20th to Jun 22nd the Cantina team conducted a review of [LayerZero-v2](#) on commit hash [9c741e7f](#). The team identified a total of **14** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	4	0	4
Gas Optimizations	0	0	0
Informational	10	0	10
Total	14	0	14

The Cantina Managed team reviewed Sky's [LayerZero-v2](#) holistically on commit hash [9c741e7f](#) and concluded that all findings were addressed and no new vulnerabilities were identified.

2.1 Scope

The security review had the following components in scope for [LayerZero-v2](#) on commit hash [9c741e7f](#):

```
packages/layerzero-v2/evm/messagelib/contracts/uln/dvn/adapters/CCIP
```

3 Findings

3.1 Low Risk

3.1.1 LayerZero confirmation guarantees for CCIP-backed DVNs reduce to Chainlink CCIP finality assumptions

Severity: Low Risk

Context: `DVNAdapterBase.sol#L88`

Description: `CCIPDVNAdapter` receives the LayerZero `UlnConfig.confirmations` value in `AssignJobParam.confirmations`, but does not enforce it. After CCIP delivery, `DVNAdapterBase._decodeAndVerify(...)` always calls the destination receive library with `MAX_CONFIRMATIONS = type(uint64).max`.

Therefore, CCIP-backed DVN votes satisfy any LayerZero confirmation requirement. The effective finality guarantee for those votes is "CCIP delivered the message", not "the source packet was observed after `UlnConfig.confirmations` source-chain confirmations".

Chain	CCIP finality
Base	Finality tag, ~18 min
Ethereum	Finality tag, ~15 min
Avalanche	Finality tag, <1s
Optimism / OP	Finality tag, ~20 min

Source: [Chainlink CCIP execution latency](#)

Recommendation: Explicitly document this trust boundary and configure receive-side DVN thresholds so CCIP-backed votes cannot satisfy quorum unless this CCIP finality assumption is acceptable.

If LayerZero confirmation parity is required, the adapter should only verify with a confirmation count that is actually enforced by the CCIP-backed path.

Sky: Acknowledged. LZ code design choice. Sky side is aware (it was discussed and commented already in https://github.com/sky-ecosystem/lz-dvn-broadcaster/pull/1#discussion_r3308481698).

Cantina Managed: Acknowledged.

3.1.2 Require prefunding the `CCIPDVNAdapter`

Severity: Low Risk

Context: `CCIPDVNAdapter.sol#L64`

Description: `SendUln302` calls the payable CCIP DVN adapter's `assignJob` without forwarding fees. DVN fee accounting is only updated after the call, so the adapter must already hold sufficient native funds for the CCIP payment to succeed.

Recommendation: Prefund the CCIP DVN adapter with sufficient native ETH before use and document this requirement.

Sky: Acknowledged. LZ code issue, Sky side is aware. The need for pre-funding was mentioned to the auditors before the audit.

Cantina Managed: Acknowledged.

3.1.3 Use of deprecated `EVMEExtraArgsV1` for CCIP messages

Severity: Low Risk

Context: `CCIPDVNAdapter.sol#L141`

Description: The adapter uses deprecated `EVMEExtraArgsV1` arguments:

```
Client._argsToBytes(Client.EVMExtraArgsV1({ gasLimit: _gas, strict: false })))
```

The current live fee quoter ignores the `strict` boolean:

EVMExtraArgsV1 originally included a second boolean (strict) field which has been deprecated.

```
function _parseUnvalidatedEVMExtraArgsFromBytes(
    bytes calldata extraArgs,
    uint64 defaultTxGasLimit
) private pure returns (Client.GenericExtraArgsV2 memory) {
    if (extraArgs.length < 4) {
        // If extra args are empty, generate default values.
        return Client.GenericExtraArgsV2({gasLimit: defaultTxGasLimit,
        ↪ allowOutOfOrderExecution: true});
    }

    bytes4 extraArgsTag = bytes4(extraArgs);
    bytes memory argsData = extraArgs[4:];

    if (extraArgsTag == Client.GENERIC_EXTRA_ARGS_V2_TAG) {
        Client.GenericExtraArgsV2 memory genericExtraArgs = abi.decode(argsData,
        ↪ (Client.GenericExtraArgsV2));
        genericExtraArgs.allowOutOfOrderExecution = true;
        return genericExtraArgs;
    } else if (extraArgsTag == Client.EVM_EXTRA_ARGS_V1_TAG) {
        // EVMExtraArgsV1 originally included a second boolean (strict) field which has been
        ↪ deprecated.
        // Clients may still include it but it will be ignored.
        return Client.GenericExtraArgsV2({gasLimit: abi.decode(argsData, (uint256)),
        ↪ allowOutOfOrderExecution: true});
    }
    revert InvalidExtraArgsTag();
}
```

In practice, v1 args are mapped to `GenericExtraArgsV2` with `allowOutOfOrderExecution = true`, which appears to be the desired behavior for DVN verification messages, but relying on v1 is outdated and inconsistent with current Chainlink documentation.

Recommendation: Consider updating CCIP message construction to use `GenericExtraArgsV2` explicitly, as shown in the [Chainlink CCIP docs](#), setting the intended gas limit and `allowOutOfOrderExecution` value.

Sky: Acknowledged. LZ code issue.

Cantina Managed: Acknowledged.

3.1.4 Source-side `CCIPDVNAdapter` makes CCIP mandatory for every packet emission

Severity: Low Risk

Context: `SendUlnBase.sol#L66-L81`, `CCIPDVNAdapter.sol#L61-L91`

Description: The `CCIPDVNAdapter` will be included in `optionalDVNs` list. In LayerZero ULN, `optionalDVNs` affects the receive-side quorum model, but the source send library still synchronously calls `assignJob()` on every configured required and optional DVN before the packet is emitted.

```
uint8 dvnsLength = _ulnConfig.requiredDVNCount + _ulnConfig.optionalDVNCount;
for (uint8 i = 0; i < dvnsLength; ++i) {
    address dvn = i < _ulnConfig.requiredDVNCount
        ? _ulnConfig.requiredDVNs[i]
        : _ulnConfig.optionalDVNs[i - _ulnConfig.requiredDVNCount];
```

```

    dvnFees[i] = ILayerZeroDVN(dvn).assignJob(_param, options);
}

```

`CCIPDVNAdapter.assignJob()` is not a simple view function, it builds a CCIP message, gets a Chainlink fee quote, checks adapter balance, and calls `router.ccipSend()` before returning:

```

(ccipFee, totalFee) = ICCIPDVNAdapterFeeLib(workerFeeLib).getFeeOnSend(
    feeLibParam,
    config,
    message,
    _options,
    ccipRouter
);

_assertBalanceAndWithdrawFee(msg.sender, ccipFee);

ccipRouter.ccipSend{ value: ccipFee }(config.chainSelector, message);

```

So any revert inside the CCIP adapter path reverts the entire LayerZero send: adapter underfunding, missing `receiveLibs` config, unsupported adapter options, malformed or unsupported CCIP lane config, CCIP fee quote failure, Chainlink router health failure, or `ccipSend()` reverting.

The verified Chainlink router uses `whenHealthy` on `ccipSend()`:

```

function ccipSend(
    uint64 destinationChainSelector,
    Client.EVM2AnyMessage memory message
) external payable whenHealthy returns (bytes32) {

```

and the live onramp fee path can revert if RMN marks the destination lane cursed:

```

if (i_rmnRemote.isCursed(bytes16(uint128(destChainSelector)))) revert
    ↳ CursedByRMN(destChainSelector);

```

Recommendation: Document the actual source-side liveness property:

CCIP is required for `source` packet emission, so `CCIPDVNAdapter.assignJob()` must not revert.

Because governance sends are executed through the Sky spell system, this check should be part of spell review and pre-execution checks for any spell that does a LayerZero message configured with this adapter.

Sky: Acknowledged. LZ code issue. The DVN will be tested as part of setting it up. Although not ideal ofc, if reverts happen governance can replace the DVN.

Cantina Managed: Acknowledged.

3.2 Informational

3.2.1 DVN adapter vs OApp receive library misconfiguration

Severity: Informational

Context: [DVNAdapterBase.sol#L44](#)

Description: `DVNAdapterBase` maintains its own `(sendLib, dstEid) → receiveLib` mapping. If this configured receive library does not align with the destination OApp's configured receive library, proper OApp message verification can fail at the destination.

Note that in Sky Gov's case of a custom broadcast receive library, the DVN adapter's `receiveLib` and the destination OApp's `receiveLib` are intentionally different.

Recommendation: Add clear documentation and/or configuration validation for the adapter receive library mapping. Where Sky intentionally uses the broadcast receive library, document that it must be

configured as the Broadcaster receive library. Otherwise, it should match the destination OApp's receive library.

Sky: Acknowledged. LZ code issue, Sky side is aware. As with other sky modules configuration is assumed trusted and handled carefully.

Cantina Managed: Acknowledged.

3.2.2 DVN fees can be discounted below cost breaking bridging longterm

Severity: Informational

Context: CCIPDVNAdapterFeeLib.sol#45

**Description:* If `multiplierBps` is set below `BPS_DENOMINATOR`, the adapter may charge `totalFee` less than the `ccipFee` paid to the router, causing insolvency longterm. This breaks the invariant that if `CCIPDVNAdapter.assignJob(...)` is called by `SendUln302`, then after the whole send path finishes, the adapter's withdrawable value `adapter.balance + SendUln302.fees[adapter]` should not decrease.

Recommendation: Ensure fees are never discounted below cost. In `worker.setDefaultMultiplierBps(fee)`, require `fee >= 1e4`. In `CCIPDVNAdapter.setDstConfig(param)`, require `param.multiplierBps == 0 || param.multiplierBps >= 1e4`.

Sky: Acknowledged. LZ code issue, Sky side is aware. As with other sky modules configuration is assumed trusted and handled carefully.

Cantina Managed: Acknowledged.

3.2.3 Avoid bidirectional setup for one-way Sky Gov bridge

Severity: Informational

Context: CCIPDVNAdapter.sol#L49-L50

Description: The adapter setup currently configures peers bidirectionally. Sky Gov only requires the L1 → L2 direction, so configuring the reverse L2 → L1 path is unnecessary for this use case and increases the enabled bridge surface.

Recommendation: Configure only the required L1 → L2 path for Sky Gov. Leave the L2 → L1 direction disabled, e.g. by not setting a `workerFeeLib` on L2 or otherwise explicitly bricking the reverse path.

Sky: Acknowledged. Sky side is aware.

Cantina Managed: Acknowledged.

3.2.4 Implicit fee quote assumption on message byte contents

Severity: Informational

Context: CCIPDVNAdapter.sol#L121, CCIPDVNAdapter.sol#L76-L77

Description: The view `getFee` fee quoter function uses an empty `message.data`, while `assignJob` quotes the fee with the actual `message`. This assumes `workerFeeLib` /CCIP Router pricing depends only on message length and not the actual bytes of `message.data`. The currently live CCIP fee quoter on Ethereum mainnet indeed only prices based on length, but this behavior could change in future router/quoter versions or differ across deployments.

Recommendation: Document the requirement that the configured CCIP fee quoter must not price based on the byte contents of `message.data`, only on its length.

Sky: Acknowledged. LZ code issue.

Cantina Managed: Acknowledged.

3.2.5 CCIPDVNAdapterFeeLib ignores configured floorMarginUSD

Severity: Informational

Context: CCIPDVNAdapterFeeLib.sol#L23-L70, DVNFeeLib.sol#L207-L224

Description: CCIPDVNAdapterFeeLib exposes floorMarginUSD in its destination config, but the value is never used during fee calculation.

The config setter stores only floorMarginUSD :

```
function setDstConfig(DstConfigParam[] calldata _param) external onlyOwner {
    for (uint256 i = 0; i < _param.length; i++) {
        DstConfigParam calldata param = _param[i];

        dstConfig[param.dstEid] = DstConfig({ floorMarginUSD: param.floorMarginUSD });
    }
}
```

However, both getFeeOnSend() and getFee() compute the total fee only from the Chainlink CCIP fee and the multiplier:

```
ccipFee = _router.getFee(_dstConfig.chainSelector, _message);
totalFee = _applyPremium(_dstConfig.multiplierBps, _params.defaultMultiplierBps,
    ↪ ccipFee);
```

```
function _applyPremium(
    uint16 _multiplierBps,
    uint16 _defaultMultiplierBps,
    uint256 _fee
) internal pure returns (uint256 fee) {
    uint256 multiplier = _multiplierBps == 0 ? _defaultMultiplierBps : _multiplierBps;
    fee = (_fee * multiplier) / BPS_DENOMINATOR;
}
```

By comparison, the normal DVNFeeLib applies floorMarginUSD as a minimum margin over the base fee. This is useful on cheap routes where a basis-point multiplier can round down or produce too small a fee:

```
uint256 feeWithMultiplier = (_fee * multiplierBps) / 10000;
if (_nativePriceUSD == 0 || _marginUSD == 0) {
    return feeWithMultiplier;
}

uint256 feeWithFloorMargin = _fee + (_marginUSD * nativeDecimalsRate) / _nativePriceUSD;

return feeWithFloorMargin > feeWithMultiplier ? feeWithFloorMargin : feeWithMultiplier;
```

Recommendation: Remove the dead fee-lib destination config surface entirely.

Sky: Acknowledged. LZ code issue.

Cantina Managed: Acknowledged.

3.2.6 CCIPDVNAdapter normalizes endpoint ids and aliases v1/v2 route configuration

Severity: Informational

Context: CCIPDVNAdapter.sol#L37-L55, CCIPDVNAdapter.sol#L74, DVNAdapterBase.sol#L83-L88

Description: CCIPDVNAdapter normalizes LayerZero endpoint IDs with % 30000 when storing and reading route configuration:

```
uint32 eid = param.eid % 30000;
```

The first config written for a normalized EID pins the Chainlink chain selector and peer bytes:

```

if (dstConfig[eid].chainSelector == 0) {
    dstConfig[eid].chainSelector = param.chainSelector;
    dstConfig[eid].peer = param.peer;
    srcConfig[param.chainSelector].eid = eid;
    srcConfig[param.chainSelector].peer = param.peer;
}

dstConfig[eid].multiplierBps = param.multiplierBps;
dstConfig[eid].gas = param.gas;

```

This appears to be legacy compatibility for using one adapter across Endpoint V1 and Endpoint V2, but it creates route aliasing. EIDs such as 101 and 30101, or 184 and 30184, share the same adapter `dstConfig` and `srcConfig` bucket.

Recommendation: The adapter is intended for Endpoint V2 routes only, remove `% 30000` normalization and key `dstConfig` and `srcConfig` by the exact EID used by the send and receive libraries.

Sky: Acknowledged. LZ code issue.

Cantina Managed: Acknowledged.

3.2.7 CCIP service limits can revert adapter sends

Severity: Informational

Context: `CCIPDVNAdapter.sol#L132-L143`, `CCIPDVNAdapterFeeLib.sol#L34-L59`, CCIP Service Limits (EVM)

Description: Unlike the current LayerZero DVNs, `CCIPDVNAdapter` drags Chainlink's CCIP service limits into the LayerZero send path. During source-side `assignJob()` it calls Chainlink `router.getFee()` and `router.ccipSend()`, so packet emission now sits behind Chainlink's limits.

The CCIP message is fixed: empty `tokenAmounts`, native fee token, and a 145-byte payload (receiveLib 32 + payloadHash 32 + packetHeader 81). So data, token-count, and token-pool limits never apply.

Chainlink's EVM service limits against adapter behavior:

CCIP restriction	Limit	CCIPDVNAdapter
Message data length	30 KB	Fixed 145 bytes, far below
Message execution gas	3,000,000 (CORN lanes 500,000)	<code>dstConfig[eid].gas</code> becomes <code>extraArgs.gasLimit</code>
Token count	1 max	<code>tokenAmounts</code> empty
Token pool execution gas	90,000	No token path

We don't hit the data/token caps. The only ones that can bite are gas limit, all driven by route config:

1. `dstConfig[eid].gas` exceeds the lane's `maxPerMsgGasLimit`
2. `dstConfig[eid].gas` too low for destination `ccipReceive()` → `verify()`. The LZ send succeeds and the packet is emitted, but the CCIP message runs out of gas at the destination OffRamp and needs `manuallyExecute` to retry.

Recommendation: Document this into a checklist for every `CCIPDVNAdapter` route:

- Keep `dstConfig[eid].gas` below the live `maxPerMsgGasLimit`.
- Measure destination `ccipReceive()` → `verify()` gas and add a bounded buffer.

Plus the verifications from finding "Source-side `CCIPDVNAdapter` makes CCIP mandatory for every packet emission".

Sky: Acknowledged. LZ code issue. Messages should be tested either during the DVN setup or with a replicated system.

Cantina Managed: Acknowledged.

3.2.8 CCIPDVNAdapter reverts on any DVN-targeted option

Severity: Informational

Context: [SendUlnBase.sol#L64-L81](#), [CCIPDVNAdapterFeeLib.sol#L34-L59](#)

Description: LayerZero options are split by worker. The send library separates executor options from DVN options, groups the DVN options by index (zero-based across `requiredDVNs` then `optionalDVNs`), and passes each DVN only the bytes targeting its index. The adapter forwards those bytes to its fee lib on both the quote and send paths, and the fee lib rejects any non-empty options:

```
if (_options.length > 0) revert CCIPDVNAdapter_OptionsUnsupported();
```

So if a DVN option is set for the adapter's index, every `quoteSend()` and `send()` for that `(dstEid, msgType)` reverts before the packet is emitted. Options reach the adapter from two places: enforced options (`setEnforcedOptions`) or a caller's per-message `extraOptions`. Either one carrying a DVN option (e.g. `addDVNPreCrimeOption(dvnIdx)`) at the adapter's index bricks the route.

Recommendation: Document the fact that `CCIPDVNAdapter` is incompatible with DVN-targeted options.

Sky: Acknowledged. There is no plan to add options directed at this DVN. The Sky Oapps are agnostic to the specific DVN set.

Cantina Managed: Acknowledged.

3.2.9 CCIP adapter pauser does not pause any CCIP adapter flow

Severity: Informational

Context: [Worker.sol#L83-L91](#), [\[CCIPDVNAdapter.sol#L61-L125\]](#)

Description: `CCIPDVNAdapter` inherits `Worker`, so the `DEFAULT_ADMIN_ROLE` can call `setPaused(true)` and make `paused()` return true. However, the pause flag is not consumed by any CCIP adapter entrypoint.

`Worker.setPaused()` is explicitly documented as pausing worker execution only "if used with `whenNotPaused`". The CCIP adapter never uses `whenNotPaused`:

Recommendation: Document this behavior in the CCIP Adapter docs

Sky: Acknowledged. No pausing for the DVN is required from our side. So we can use the unchanged `CCIPDVNAdapter` code.

Cantina Managed: Acknowledged.

3.2.10 Minor issues, Typos & Documentation

Severity: Informational

Context: (See each case below)

Description:

1. [DVNAdapterBase.sol#L35](#): `RecieveLib` in `DVNAdapter_MissingRecieveLib` is misspelled and should be `ReceiveLib`.

Recommendation: Consider addressing the issues mentioned above.

Sky: Acknowledged. LZ code issue. Typo is acknowledged by Sky side.

Cantina Managed: Acknowledged.